Министерство науки и высшего образования РФ
Пермский национальный исследовательский политехнический университет
Электротехнический факультет
Кафедра Информационные технологии и автоматизированные системы

Базы данных

Лабораторная работа № 4

Тема: «Реализация команд DDL, DML и соединений на веб-странице»

Выполнил: студент
группы РИС-25-2б
Ильинов Ф. М.
Проверил: доцент кафедры ИТАС
Петренко А.А.

г. Пермь – 2026

СОДЕРЖАНИЕ

1 Цель работы 4

2 Задачи работы 5

3 Этапы выполнения 6

3.1 Архитектура веб-приложения и стек вызовов 7

3.1.1 Веб-сервер и маршрутизация запросов 7

3.1.2 Архитектура обработчиков и обработка ошибок 9

3.1.3 Движок представления и шаблонизация 11

3.1.4 Пул соединений и взаимодействие с PostgreSQL 13

3.1.5 Механизм безопасной сборки SQL-запросов 14

3.2 Динамическое получение метаданных PostgreSQL и просмотр таблиц 16

3.2.1 Чтение системного каталога information_schema 16

3.2.2 GET /tables 17

3.3 Реализация команд DDL 21

3.3.1 Создание таблиц: POST /ddl/create 21

3.3.2 Модификация схемы: POST /ddl/alter 24

3.3.3 Очистка и удаление таблиц: TRUNCATE и DROP 26

3.4 Реализация команд DML 28

3.4.1 Пакетная вставка записей: POST /dml/insert 28

3.4.2 Модификация данных: стек вызова POST /dml/update 31

3.4.3 Удаление данных: стек вызова POST /dml/delete 32

3.5 Реализация выборок и соединений (DQL) 33

3.5.1 Параметризованная выборка: POST /query/select 33

3.5.2 Внутреннее соединение таблиц: POST /query/join 35

3.6 Механизм сброса и начальной инициализации схемы 37

3.6.1 Начальная инициализация схемы базы данных 37
3.6.2 Сброс POST /reset 39
3.7 Результаты работы и снимки экрана приложения 41
Заключение 47
Список использованных источников 48

1 Цель работы

Реализовать веб-приложение для взаимодействия с СУБД PostgreSQL, обеспечивающее выполнение полного жизненного цикла операций с реляционной базой данных:

а) Команды DDL: создание, модификация, очистка и удаление таблиц;

б) Команды DML: вставка, обновление и удаление;

в) Команды SELECT и INNER JOIN ... ON);

2 Задачи работы

В соответствии с целью были поставлены следующие задачи:

а) Разработать локальный веб-сервер на языке Golang (версия 1.27) с использованием стандартной библиотеки `net/http`;

б) Реализовать подключение к СУБД PostgreSQL с использованием пула соединений `pgx`;

в) Разработать механизм безопасной трансляции параметров пользовательских форм в команды SQL с экранированием;

г) Реализовать стек вызовов для команд DDL;

д) Реализовать стек вызовов для команд DML;

е) Реализовать стек вызовов для выборок и соединений: построитель запросов SELECT и оператор соединения INNER JOIN по ключевым полям;

ж) Разработать веб-интерфейс на основе Server-Side Rendering (пакет `html/template`) с выводом результатов и текста выполненного SQL-запроса на страницу браузера.

3 Этапы выполнения

Для выполнения поставленных задач были сформулированы следующие технологические требования к инструментарию:

– Поддержка реляционной модели данных и стандарта SQL в СУБД PostgreSQL [1, 2];

– Поддержка сетевого протокола передачи гипертекста HTTP [3];

– Возможность отрисовки форм, кнопок и динамических таблиц в веб-интерфейсе.

Для реализации клиентского интерфейса был выбран язык разметки HTML [4] совместно с библиотекой стилей Pico CSS [5], обеспечивающей лаконичное визуальное оформление без необходимости написания сложного клиентского JavaScript-кода.

В качестве языка серверной части выбран Golang версии 1.27 [6]. Для взаимодействия с СУБД PostgreSQL задействован драйвер и пул соединений `github.com/jackc/pgx` [7].

Сервер функционирует по принципу Server-Side Rendering (SSR). Все страницы собираются на стороне сервера с помощью встроенного шаблонизатора `html/template` и возвращаются браузеру в виде готового HTML-документа.

3.1 Архитектура веб-приложения и стек вызовов

Веб-приложение построено по многоуровневой схеме. Прохождение любого пользовательского действия организовано в виде строгого сквозного стека вызовов:

а) клиентский уровень: веб-страница браузера (HTML-формы с параметрами операций DDL, DML или DQL);

б) уровень веб-сервера и маршрутизации: маршрутизатор стандартной библиотеки Golang (версия 1.27), диспетчеризация по HTTP-методу и пути, распаковка параметров формы;

в) уровень сборщика запросов (Query Builder): экранирование идентификаторов PostgreSQL двойными кавычками (`quoteIdent`) и строковых литералов одинарными кавычками (`quoteLiteral`);

г) уровень СУБД PostgreSQL: выполнение запроса через пул соединений `pgx`, опрос системного каталога `information_schema`;

д) уровень представления (View Renderer): динамическое сканирование кортежей и отрисовка HTML-шаблона с отображением текста выполненного SQL-запроса и полученных данных.

3.1.1 Веб-сервер и маршрутизация запросов

Инициализация сетевого мультиплексора и регистрация маршрутов выполняются в функции `main` (листинг 1).

```
70          mux := http.NewServeMux()
71          mux.Handle("GET /css/", http.StripPrefix("/css/", cssFS))
72
73          mux.HandleFunc("GET /{$}", func(w http.ResponseWriter, r
            ↪  *http.Request) {
74              http.Redirect(w, r, "/tables", http.StatusSeeOther)
75          })
76
77          mux.Handle("GET /tables", HandleTables(views, pool))
78          mux.Handle("GET /tables/{name}", HandleTables(views, pool))
79
80          mux.HandleFunc("GET /ddl", func(w http.ResponseWriter, r
            ↪  *http.Request) {
81              http.Redirect(w, r, "/ddl/create", http.StatusSeeOther)
82          })
83          mux.Handle("GET /ddl/{op}", HandleDDL(views, pool))
84          mux.Handle("POST /ddl/create", CreateTableHandler(views, pool))
85          mux.Handle("POST /ddl/alter", AlterTableHandler(views, pool))
86          mux.Handle("POST /ddl/truncate", TruncateTableHandler(views, pool))
87          mux.Handle("POST /ddl/drop", DropTableHandler(views, pool))
88
89          mux.HandleFunc("GET /dml", func(w http.ResponseWriter, r
            ↪  *http.Request) {
90              http.Redirect(w, r, "/dml/insert", http.StatusSeeOther)
91          })
92          mux.Handle("GET /dml/{op}", HandleDML(views, pool))
93          mux.Handle("POST /dml/insert", InsertHandler(views, pool))
94          mux.Handle("POST /dml/update", UpdateHandler(views, pool))
95          mux.Handle("POST /dml/delete", DeleteHandler(views, pool))
96
97          mux.Handle("GET /select", HandleSelect(views, pool))
98          mux.Handle("GET /select/{table}", HandleSelect(views, pool))
99          mux.Handle("POST /query/select", SelectHandler(views, pool))
100
101         mux.Handle("GET /join", HandleJoin(views, pool))
102         mux.Handle("POST /query/join", JoinHandler(views, pool))
103
104         mux.Handle("POST /reset", ResetHandler(views, pool))
105
```

Листинг 1 — Маршрутизация HTTP-запросов DDL, DML и DQL в `main.go`

В строке 73 шаблон `"GET /{$}"` задает строгое соответствие корневому пути и выполняет перенаправление на страницу таблиц `/tables`. Для остальных операций предусмотрено четкое разделение:

— `GET`-запросы отображают страницы и формы (например, `/ddl/{op}`, `/dml/{op}`, `/select`);

— `POST`-запросы принимают данные форм и запускают сборку и выполнение соответствующих SQL-команд (`/ddl/create`, `/dml/insert`, `/query/select` и др.).

3.1.2 Архитектура обработчиков и обработка ошибок

Для унификации обработки ошибок используется тип-адаптер `AppHandler` (листинг 2), представляющий собой функцию, возвращающую ошибку.

```go
type AppError struct {
        Err     error
        Message string
        Code    int
}

func (e *AppError) Error() string {
        if e.Message != "" {
                return e.Message
        }
        if e.Err != nil {
                return e.Err.Error()
        }
        return "Internal Server Error"
}

func (e *AppError) Unwrap() error {
        return e.Err
}

type AppHandler func(w http.ResponseWriter, r *http.Request) error

func (fn AppHandler) ServeHTTP(w http.ResponseWriter, r *http.Request) {
        if err := fn(w, r); err != nil {
                var appErr *AppError
                if errors.As(err, &appErr) {
                        slog.Error("request error",
                                slog.String("path", r.URL.Path),
                                slog.String("message", appErr.Message),
                                slog.Any("err", appErr.Err),
                        )
                        code := appErr.Code
                        if code == 0 {
                                code = http.StatusInternalServerError
                        }
                        http.Error(w, appErr.Message, code)
                        return
                }

                slog.Error("unhandled server error", slog.String("path",
                ↪  r.URL.Path), slog.Any("err", err))
                http.Error(w, "Internal server error: "+err.Error(),
                ↪  http.StatusInternalServerError)
        }
}
```

Листинг 2 — Структура `AppError` и адаптер обработчиков `AppHandler`

Метод `ServeHTTP` перехватывает ошибки выполнения. Если возвращена ошибка `AppError`, фиксируется запись в логе и клиенту отправляется соответствующий HTTP-статус. При возникновении ошибки на уровне СУБД форма рендерится повторно со статусом `400 Bad Request`, сохраняя введенные пользователем значения и подсвечивая сообщение сервера PostgreSQL и проблемный SQL-запрос.

3.1.3 Движок представления и шаблонизация

Генерация веб-интерфейса реализована на стороне сервера с помощью встроенного пакета `html/template` и обертки `ViewRenderer` (листинг 3).

```
14 type ViewRenderer struct {
15         templates map[string]*template.Template
16 }
17
18 func NewViewRenderer(funcMap template.FuncMap) (*ViewRenderer, error) {
19         layout, err :=
           ↪  template.New("layout.html").Funcs(funcMap).ParseFiles("templates/layout.h
20         if err != nil {
21                 return nil, fmt.Errorf("parse layout.html: %w", err)
22         }
23
24         pages := []string{"tables", "ddl", "dml", "select", "join"}
25         tmpls := make(map[string]*template.Template, len(pages))
26         for _, p := range pages {
27                 t, err := layout.Clone()
28                 if err != nil {
29                         return nil, fmt.Errorf("clone layout for %s: %w",
                           ↪  p, err)
30                 }
31                 path := filepath.Join("templates", p+".html")
32                 t, err = t.ParseFiles(path)
33                 if err != nil {
34                         return nil, fmt.Errorf("parse %s: %w", path, err)
35                 }
36                 tmpls[p] = t
37         }
38         return &ViewRenderer{templates: tmpls}, nil
39 }
```

Листинг 3 — Инициализация и рендеринг шаблонов в `views.go`

При запуске сервера компилируется базовый шаблон `layout.html`, после чего методом `Clone()` создаются изолированные копии для каждой рабочей страницы (`tables`, `ddl`, `dml`, `select`, `join`). Для динамических вычислений в разметке зарегистрированы специальные функции (листинг 4).

```
        funcMap := template.FuncMap{
                "add": func(a, b int) int { return a + b },
                "seq": func(n int) []int {
                        res := make([]int, n)
                        for i := range res {
                                res[i] = i
                        }
                        return res
                },
                "eqStr": func(a, b string) bool { return a == b },
                "getRowVal": func(rows []map[string]string, rIdx int,
                ↪  colName string) string {
                        if rIdx >= 0 && rIdx < len(rows) && rows[rIdx] !=
                        ↪  nil {
                                return rows[rIdx][colName]
                        }
                        return ""
                },
        }
```

Листинг 4 — Пользовательские функции шаблонизатора `funcMap`

Функции `seq` и `add` используются для циклической генерации динамических строк формы (например, добавление заданного количества колонок в DDL или кортежей в DML). Функция `getRowVal` обеспечивает безопасное извлечение сохраненных значений полей при повторной отрисовке после ошибок.

Базовый шаблон `layout.html` (листинг 5) содержит навигацию, кнопку быстрого сброса БД и блок отображения сгенерированного SQL-запроса. Для предотвращения выхода широких таблиц за границы страницы контейнерам `figure` назначен стиль горизонтальной прокрутки `overflow-x: auto`.

```
16 <main class="container">
17     <header>
18         <article>
19             <strong>DB Laboratory Work №4</strong><br>
20             <small>RIS-25-2b Ilinov F. M.</small>
21             <form action="/reset" method="post">
22                 <button type="submit">Reset</button>
23             </form>
24         </article>
25         <article>
26             <a role="button" href="/tables">SHOW</a>
27            <a role="button" href="/ddl/create">Data Definition Language</a>
28             <a role="button" href="/dml/insert">Data Manipulation
               ↪  Language</a>
29             <a role="button" href="/select">SELECT</a>
30             <a role="button" href="/join">INNER JOIN</a>
31         </article>
32     </header>
33
34     {{ if eqStr .Status "error" }}
35     <article>
36         <header><strong>ERROR:</strong> {{ .Message }}</header>
37         {{ if .SQL }}
38         <pre><code>{{ .SQL }}</code></pre>
39         {{ end }}
40     </article>
41     {{ end }}
42
43     {{ block "content" . }}
44     {{ end }}
45
46 </main>
```

Листинг 5 — Разметка контейнера и блок аудита SQL в `layout.html`

В строках 34–41 размещен блок уведомления об ошибках: если сервер СУБД возвращает ошибку, точный текст сформированного запроса выводится в теге `<pre><code>`.

3.1.4 Пул соединений и взаимодействие с PostgreSQL

Подключение к СУБД осуществляется в функции `main` через пул соединений библиотеки `pgx` (листинг 6).

```
25          pool, err := pgx.NewConnPool(pgx.ConnPoolConfig{
26                  Host:           dbHost,
27                  Port:           dbPort,
28                  Database:       dbName,
29                  User:           dbUser,
30                  Password:       dbPassword,
31                  MaxConnections: 10,
32          })
33          if err != nil {
34                  slog.Error("cannot connect to postgres", slog.Any("err",
                    ↪  err))
35                  os.Exit(1)
36          }
37          defer pool.Close()
```

Листинг 6 — Инициализация пула соединений pgx

Пул соединений повторно использует открытые TCP-сессии к PostgreSQL, снижая накладные расходы на постоянную аутентификацию и установление соединения при каждом HTTP-запросе.

3.1.5 Механизм безопасной сборки SQL-запросов

Поскольку имена таблиц и столбцов передаются из веб-интерфейса динамически, прямое использование позиционных плейсхолдеров PostgreSQL (`$1, $2`) для них неприменимо. Для защиты от внедрения вредоносного SQL-кода (SQL Injection) в файле `db.go` реализованы специализированные функции экранирования (листинг 7).

```
39  func quoteIdent(name string) string {
40          return `"` + strings.ReplaceAll(name, `"`, `""`) + `"`
41  }
42
43  func quoteLiteral(val string) string {
44          return `'` + strings.ReplaceAll(val, `'`, `''`) + `'`
45  }
```

Листинг 7 — Функции экранирования идентификаторов и литералов

Функция `quoteIdent` экранирует идентификаторы PostgreSQL (имена таблиц, колонок) согласно стандарту: имя оборачивается в двойные кавыч-

ки, а внутренние кавычки удваиваются (). Функция `quoteLiteral` обрамляет пользовательские строковые константы в одинарные кавычки с удвоением внутренних одинарных кавычек.

3.2 Динамическое получение метаданных PostgreSQL и просмотр таблиц

В отличие от статических приложений, где структура таблиц зафиксирована в структурах данных языка программирования, разрабатываемый вебсервис должен уметь взаимодействовать с произвольными таблицами, создаваемыми пользователем в рантайме. Для этого реализовано динамическое получение метаданных СУБД.

3.2.1 Чтение системного каталога information_schema

Получение перечня пользовательских таблиц и описания их столбцов выполняется посредством запросов к стандартным системным представлениям `information_schema` СУБД PostgreSQL.

Для получения списка базовых таблиц публичной схемы используется SQL-запрос из файла `get_tables.sql` (листинг 8).

```
1 SELECT table_name
2 FROM information_schema.tables
3 WHERE table_schema = 'public' AND table_type = 'BASE TABLE'
4 ORDER BY table_name;
```

Листинг 8 — Запрос списка пользовательских таблиц из `information_schema.tables`

Условие `table_schema = 'public'` исключает системные таблицы СУБД, а предикат `table_type = 'BASE TABLE'` фильтрует виртуальные представления (VIEW).

Для чтения структуры выбранной таблицы используется параметризованный запрос из файла `get_columns.sql` (листинг 9).

```
SELECT column_name, data_type
FROM information_schema.columns
WHERE table_schema = CURRENT_SCHEMA AND table_name = $1
ORDER BY ordinal_position;
```

Листинг 9 — Запрос описания колонок таблицы из `information_schema.columns`

Запрос возвращает имена атрибутов и их типы данных (`data_type`), упорядоченные по физическому положению в таблице (`ordinal_position`).

3.2.2 GET /tables

Просмотр содержимого таблиц инициируется запросом по маршруту `GET /tables` (для вывода всех таблиц) или `GET /tables/{name}` (для выбранной таблицы). Логика обработки сосредоточена в функции `HandleTables` (листинг 10).

```
9  func HandleTables(views *ViewRenderer, pool *pgx.ConnPool) AppHandler {
10         return func(w http.ResponseWriter, r *http.Request) error {
11                 base, err := newBasePage(pool)
12                 if err != nil {
13                         return &AppError{Err: err, Message: "Database
                           ↪  error: " + err.Error(), Code:
                           ↪  http.StatusInternalServerError}
14                 }
15
16                 data := TablesPageData{BasePage: base}
17                 tableName := r.PathValue("name")
18                 if tableName == "" {
19                         tableName = r.URL.Query().Get("table")
20                 }
21
22                 if tableName == "" {
23                         data.ShowViewAll = true
24                         if all, err := getAllTablesData(pool); err == nil {
25                                 data.AllTables = all
26                         }
27                 } else {
28                         data.SelectedTable = tableName
29                         td, err := getTableData(pool, tableName)
30                         if err != nil {
31                                 data.Status = "error"
32                                 data.Message = "Table not found: " +
                                   ↪  err.Error()
33                                 return views.RenderStatus(w, "tables",
                                   ↪  http.StatusNotFound, data)
34                         }
35                         data.CurrentTable = td
36                 }
37
38                 return views.Render(w, "tables", data)
39         }
40 }
```

Листинг 10 — Обработчик страницы просмотра таблиц в `handlers_tables.go`

В строке 11 вызывается функция `newBasePage`, наполняющая навигационный выпадающий список актуальным перечнем таблиц из каталога СУБД. Если конкретная таблица не выбрана, вызывается `getAllTablesData`, опрашивающая каждую таблицу по очереди.

Поскольку схема возвращаемых данных заранее неизвестна, в файле db.go реализована функция scanRowsToTableData (листинг 11).

```
125 func scanRowsToTableData(rows *pgx.Rows, tableName string) (*TableData,
    ↪  error) {
126         fields := rows.FieldDescriptions()
127         colNames := make([]string, len(fields))
128         for i, f := range fields {
129                 colNames[i] = f.Name
130         }
131 
132         var dataRows [][]any
133         for rows.Next() {
134                 vals, err := rows.Values()
135                 if err != nil {
136                         return nil, err
137                 }
138                 for i, v := range vals {
139                         if t, ok := v.(time.Time); ok {
140                                 vals[i] = t.Format("2006-01-02")
141                         }
142                 }
143                 dataRows = append(dataRows, vals)
144         }
145         if err := rows.Err(); err != nil {
146                 return nil, err
147         }
148 
149         return &TableData{
150                 Name:    tableName,
151                 Columns: colNames,
152                 Rows:    dataRows,
153         }, nil
154 }
```

Листинг 11 — Динамическое чтение метаданных выборки и значений кортежей

Метод rows.FieldDescriptions() извлекает названия колонок из заголовка ответа PostgreSQL, а метод rows.Values() считывает кортеж значений в обобщенный слайс []any. Значения даты и времени (time.Time) приводятся к стандартизированному строковому виду "YYYY-MM-DD".

Полученная структура `TableData` передается в шаблон `tables.html` (листинг 12), где выполняется отрисовка HTML-таблицы.

```
            <figure>
                <table>
                    <thead>
                        <tr>
                            {{ range .Columns }}
                            <th>{{ . }}</th>
                            {{ end }}
                        </tr>
                    </thead>
                    <tbody>
                        {{ range .Rows }}
                        <tr>
                            {{ range . }}
                            <td>{{ . }}</td>
                            {{ end }}
                        </tr>
                        {{ else }}
                        <tr><td colspan="{{ len .Columns
                        ↪ }}"><em>Empty</em></td></tr>
                        {{ end }}
                    </tbody>
                </table>
            </figure>
```

Листинг 12 — Отрисовка динамической таблицы в `templates/tables.html`

Цикл `{{ range .Columns }}` генерирует заголовки таблицы (`<th>`), а вложенный цикл `{{ range .Rows }}` итерирует строки и ячейки (`<td>`).

3.3 Реализация команд DDL

Команды Data Definition Language отвечают за создание, модификацию и удаление объектов базы данных. В веб-приложении операции DDL сгруппированы по маршруту `/ddl/{op}` со следующими режимами: `create`, `alter`, `truncate`, `drop`.

3.3.1 Создание таблиц: POST /ddl/create

Процесс создания таблицы разделен на два логических шага:

а) первый шаг: пользователь указывает желаемое количество столбцов будущей таблицы;

б) второй шаг: динамически строится форма с полями конфигурации каждого столбца: наименование, тип данных PostgreSQL (`INTEGER`, `TEXT`, `BOOLEAN`, `DATE`), признак допустимости NULL значений (`Nullable`), выбор первичного ключа (`PRIMARY KEY`) и параметры внешнего ключа (`FOREIGN KEY`).

Фрагмент формы второго шага приведен в листинге 13.

```
        <article>
            <h3>CREATE TABLE</h3>
            <form action="/ddl/create" method="post">
                <input type="hidden" name="col_count" value="{{
                ↪  .DDLColCount }}">
                <label>
                    Name:
                    <input type="text" name="table_name" value="{{
                    ↪  .TableName }}" required autofocus>
                </label>

                <figure>
                    <table>
                        <thead>
                            <tr>
                                <th>№</th>
                                <th>Name</th>
                                <th>Type</th>
                                <th>Nullable</th>
                                <th>FK</th>
                                <th>ON DELETE</th>
                                <th>ON UPDATE</th>
                            </tr>
                        </thead>
```

Листинг 13 — Динамическая форма настройки колонок таблицы в `templates/ddl.html`

При отправке формы запрос поступает на эндпоинт `POST /ddl/create`, где запускается парсер параметров `parseCreateColumns` (листинг 14).

```
34 func parseCreateColumns(r *http.Request, colCount int, tableName string)
   ↪  ([]ColumnDef, []string) {
35         pkIndex, pkErr := strconv.Atoi(r.FormValue("pk_index"))
36         hasPK := pkErr == nil && pkIndex >= 0 && pkIndex < colCount
37
38         parsedCols := make([]ColumnDef, colCount)
39         var colDefs []string
40         var pkColName string
41
42         for i := 0; i < colCount; i++ {
43                 cName :=
                   ↪  strings.TrimSpace(r.FormValue(fmt.Sprintf("col_name_%d",
                   ↪  i)))
44                 cType :=
                   ↪  strings.TrimSpace(r.FormValue(fmt.Sprintf("col_type_%d",
                   ↪  i)))
45                 if cType == "" {
46                         cType = "TEXT"
47                 }
48                 isNullable := r.FormValue(fmt.Sprintf("col_nullable_%d",
                   ↪  i)) == "1"
49                 fkTable :=
                   ↪  strings.TrimSpace(r.FormValue(fmt.Sprintf("fk_table_%d",
                   ↪  i)))
50                 fkColRaw :=
                   ↪  strings.TrimSpace(r.FormValue(fmt.Sprintf("fk_col_%d",
                   ↪  i)))
51                 fkSelfIdx := -1
52
```

Листинг 14 — Парсинг полей и первичного ключа в `handlers_ddl.go`

Особое внимание уделено поддержке самоссылающихся внешних ключей (префикс `__self__`), когда столбец ссылается на первичный ключ этой же создаваемой таблицы.

После формирования списка столбцов обработчик `CreateTableHandler` формирует итоговый SQL-запрос и передает его СУБД (листинг 15).

```
228                 if tableName == "" {
229                         return renderErr("Table name cannot be empty", "")
230                 }
231                 if len(colDefs) == 0 {
232                         return renderErr("At least one column must be
                            ↪  specified", "")
233                 }
234
235                 querySQL := fmt.Sprintf("CREATE TABLE %s (\n    %s\n);",
                    ↪  quoteIdent(tableName), strings.Join(colDefs, ",\n
                    ↪  "))
236                 if _, err := pool.Exec(querySQL); err != nil {
237                         return renderErr(fmt.Sprintf("Failed to create
                            ↪  table (CREATE TABLE): %v", err), querySQL)
238                 }
239
240                 redirectSuccess(w, r, fmt.Sprintf("/tables/%s",
                    ↪  url.PathEscape(tableName)))
241                 return nil
242         }
243 }
```

Листинг 15 — Сборка и выполнение запроса `CREATE TABLE`

В случае успешного исполнения запроса клиент перенаправляется на страницу созданной таблицы `/tables/{name}`. При ошибке форма отображается повторно с сохранением введенных данных и выводом сообщения сервера СУБД.

3.3.2 Модификация схемы: POST /ddl/alter

Операция изменения структуры таблицы реализуется обработчиком `AlterTableHandler` (листинг 16).

```
277                 var querySQL string
278                 switch action {
279                 case "add":
280                         if colType == "" {
281                                 colType = "TEXT"
282                         }
283                         querySQL = fmt.Sprintf("ALTER TABLE %s ADD COLUMN
                            ↪  %s %s;", quoteIdent(table),
                            ↪  quoteIdent(effectiveColName), colType)
284                 case "alter_type":
285                         if colType == "" {
286                                 colType = "TEXT"
287                         }
288                         querySQL = fmt.Sprintf("ALTER TABLE %s ALTER COLUMN
                            ↪  %s TYPE %s USING %s::%s;", quoteIdent(table),
                            ↪  quoteIdent(effectiveColName), colType,
                            ↪  quoteIdent(effectiveColName), colType)
289                 case "drop":
290                         querySQL = fmt.Sprintf("ALTER TABLE %s DROP COLUMN
                            ↪  %s CASCADE;", quoteIdent(table),
                            ↪  quoteIdent(effectiveColName))
291                 default:
292                         return renderErr("Unknown ALTER action", "")
293                 }
294
295                 if _, err := pool.Exec(querySQL); err != nil {
296                         return renderErr(fmt.Sprintf("Failed to execute
                            ↪  ALTER TABLE: %v", err), querySQL)
297                 }
298
299                 redirectSuccess(w, r, fmt.Sprintf("/tables/%s",
                    ↪  url.PathEscape(table)))
300                 return nil
301         }
302 }
```

Листинг 16 — Сборка и исполнение команд `ALTER TABLE`

Поддерживаются три сценария:

а) добавление нового столбца;

б) изменение типа существующего столбца;

в) удаление столбца.

3.3.3 Очистка и удаление таблиц: TRUNCATE и DROP

Для быстрого удаления всех записей таблицы без разрушения ее схемы используется обработчик `TruncateTableHandler` (листинг 17).

```
304 func TruncateTableHandler(views *ViewRenderer, pool *pgx.ConnPool)
    ↪  AppHandler {
305         return func(w http.ResponseWriter, r *http.Request) error {
306                 if err := r.ParseForm(); err != nil {
307                         return &AppError{Err: err, Message: "Invalid form
                            ↪  data", Code: http.StatusBadRequest}
308                 }
309
310                 table := strings.TrimSpace(r.FormValue("table"))
311                 if table == "" {
312                         return renderDDLError(w, views, pool, "truncate",
                            ↪  1, "", "Select a table to truncate", "", nil)
313                 }
314
315                 querySQL := fmt.Sprintf("TRUNCATE TABLE %s CASCADE;",
                    ↪  quoteIdent(table))
316                 if _, err := pool.Exec(querySQL); err != nil {
317                         return renderDDLError(w, views, pool, "truncate",
                            ↪  1, table, fmt.Sprintf("Failed to execute
                            ↪  TRUNCATE: %v", err), querySQL, nil)
318                 }
319
320                 redirectSuccess(w, r, fmt.Sprintf("/tables/%s",
                    ↪  url.PathEscape(table)))
321                 return nil
322         }
323 }
```

Листинг 17 — Сборка и исполнение команды `TRUNCATE TABLE`

Удаление сущности целиком выполняется обработчиком `DropTableHandler` (листинг 18).

```
325 func DropTableHandler(views *ViewRenderer, pool *pgx.ConnPool) AppHandler {
326         return func(w http.ResponseWriter, r *http.Request) error {
327                 if err := r.ParseForm(); err != nil {
328                         return &AppError{Err: err, Message: "Invalid form
                            ↪  data", Code: http.StatusBadRequest}
329                 }
330
331                 table := strings.TrimSpace(r.FormValue("table"))
332                 if table == "" {
333                         return renderDDLError(w, views, pool, "drop", 1,
                            ↪  "", "Select a table to drop", "", nil)
334                 }
335
336                 querySQL := fmt.Sprintf("DROP TABLE %s CASCADE;",
                    ↪  quoteIdent(table))
337                 if _, err := pool.Exec(querySQL); err != nil {
338                         return renderDDLError(w, views, pool, "drop", 1,
                            ↪  table, fmt.Sprintf("Failed to execute DROP
                            ↪  TABLE: %v", err), querySQL, nil)
339                 }
340
341                 redirectSuccess(w, r, "/tables")
342                 return nil
343         }
344 }
```

Листинг 18 — Сборка и исполнение команды `DROP TABLE`

Указание ключевого слова `CASCADE` предотвращает блокировку операции СУБД в случаях, когда на удаляемую таблицу ссылаются внешние ключи других отношений.

3.4 Реализация команд DML

Команды Data Manipulation Language предназначены для добавления, изменения и удаления кортежей. В приложении поддерживаются три базовые операции: `INSERT INTO`, `UPDATE` и `DELETE`.

3.4.1 Пакетная вставка записей: POST /dml/insert

Добавление кортежей организовано в два шага:

а) первый шаг: пользователь выбирает целевую таблицу из выпадающего списка и указывает количество добавляемых строк;

б) второй шаг: сервер генерирует табличную сетку ввода, где каждый столбец соответствует реальному атрибуту таблицы из `information_schema`, а строки нумеруются функцией `seq`.

Фрагмент генерации сетки ввода приведен в листинге 19.

```
                <figure>
                    <table>
                        <thead>
                            <tr>
                                <th>#</th>
                                {{ range .TableColumns }}
                                <th>
                                    {{ .Name }} {{ .DataType }}

                                </th>
                                {{ end }}
                            </tr>
                        </thead>
                        <tbody>
                            {{ range $r := seq .DMLRowCount }}
                            <tr>
                                <td><strong>{{ add $r 1 }}</strong></td>
                                {{ range $.TableColumns }}
                                <td>
                                   <input type="text" name="val_{{ $r }}_{{
                                     ↪  .Name }}" value="{{ getRowVal
                                     ↪  $.InsertRows $r .Name }}"
                                     ↪  placeholder="{{ .Name }}">
                                </td>
                                {{ end }}
                            </tr>
                            {{ end }}
                        </tbody>
                    </table>
                </figure>
```

Листинг 19 — Генерация динамической сетки ввода кортежей в `templates/dml.html`

При отправке данных обработчик `InsertHandler` вызывает функцию `parseInsertRows` (листинг 20), формирующую списки значений.

```
func parseInsertRows(r *http.Request, cols []string, rowCount int)
↪  ([]map[string]string, []string) {
        insertRows := make([]map[string]string, rowCount)
        var rowTuples []string

        for rIdx := 0; rIdx < rowCount; rIdx++ {
                rowMap := make(map[string]string, len(cols))
                var valLiterals []string
                for _, c := range cols {
                        val := r.FormValue(fmt.Sprintf("val_%d_%s", rIdx,
                        ↪  c))
                        rowMap[c] = val
                        if val == "" {
                                valLiterals = append(valLiterals, "NULL")
                        } else {
                                valLiterals = append(valLiterals,
                                ↪  quoteLiteral(val))
                        }
                }
                insertRows[rIdx] = rowMap
                rowTuples = append(rowTuples, "("+strings.Join(valLiterals,
                ↪  ", ")+")")
        }
        return insertRows, rowTuples
}
```

Листинг 20 — Парсинг многострочных кортежей и экранирование в `handlers_dml.go`

Если значение ячейки оставлено пустым, оно преобразуется в ключевое слово `NULL`. Непустые значения экранируются функцией `quoteLiteral`. В результате формируется пакетный SQL-запрос с перечислением нескольких кортежей через запятую (листинг 21).

```
148                 querySQL := fmt.Sprintf("INSERT INTO %s (%s) VALUES\n%s;",
149                         quoteIdent(table),
150                         strings.Join(quotedCols, ", "),
151                         strings.Join(rowTuples, ",\n"))
152
153                 if _, err := pool.Exec(querySQL); err != nil {
154                         return renderErr(fmt.Sprintf("Failed to execute
                            ↪  INSERT INTO: %v", err), querySQL)
155                 }
156
157                 redirectSuccess(w, r, fmt.Sprintf("/tables/%s",
                    ↪  url.PathEscape(table)))
158                 return nil
159         }
160 }
```

Листинг 21 — Сборка и выполнение пакетного запроса `INSERT INTO`

3.4.2 Модификация данных: стек вызова POST /dml/update

Изменение данных реализуется обработчиком `UpdateHandler` (листинг 22).

```
189                 querySQL := fmt.Sprintf("UPDATE %s SET %s = %s%s;",
190                         quoteIdent(table),
191                         quoteIdent(setCol),
192                         quoteLiteral(setVal),
193                         buildSimpleWhere(whereCol, whereOp, whereVal))
194
195                 if _, err := pool.Exec(querySQL); err != nil {
196                         return renderErr(fmt.Sprintf("Failed to execute
                            ↪  UPDATE: %v", err), querySQL)
197                 }
198
199                 redirectSuccess(w, r, fmt.Sprintf("/tables/%s",
                    ↪  url.PathEscape(table)))
200                 return nil
201         }
202 }
```

Листинг 22 — Конструирование и выполнение команды `UPDATE`

Пользователь указывает имя таблицы, целевую колонку, новое строковое значение, а также условие фильтрации. Условия WHERE конструируется вспомогательной функцией `buildSimpleWhere` (листинг 23).

```
156 func buildSimpleWhere(col, op, val string) string {
157         if col == "" || op == "" {
158                 return ""
159         }
160         if op == "IS NULL" || op == "IS NOT NULL" {
161                 return fmt.Sprintf(" WHERE %s %s", quoteIdent(col), op)
162         }
163         return fmt.Sprintf(" WHERE %s %s %s", quoteIdent(col), op,
            ↪  quoteLiteral(val))
164 }
```

Листинг 23 — Конструирование предиката WHERE в `db.go`

3.4.3 Удаление данных: стек вызова POST /dml/delete

Удаление строк выполняется обработчиком `DeleteHandler` (листинг 24).

```
227                 querySQL := fmt.Sprintf("DELETE FROM %s%s;",
                    ↪  quoteIdent(table), buildSimpleWhere(whereCol, whereOp,
                    ↪  whereVal))
228
229                 if _, err := pool.Exec(querySQL); err != nil {
230                         return renderErr(fmt.Sprintf("Failed to execute
                            ↪  DELETE: %v", err), querySQL)
231                 }
232
233                 redirectSuccess(w, r, fmt.Sprintf("/tables/%s",
                    ↪  url.PathEscape(table)))
234                 return nil
235         }
236 }
```

Листинг 24 — Конструирование и выполнение команды `DELETE FROM`

3.5 Реализация выборок и соединений (DQL)

В приложении представлены два ключевых механизма: `SELECT` и `INNER JOIN`.

3.5.1 Параметризованная выборка: POST /query/select

Конструктор выборки `/select` позволяет пользователю интерактивно настраивать три компонента SQL-запроса:

а) проекция колонок;

б) предикаты фильтрации WHERE;

в) сортировка ORDER BY.

Фрагмент формы настройки выборки представлен в листинге 25.

```
        <figure>
            <table>
                <thead>
                    <tr>
                        <th>NAME</th>
                        <th>Is included</th>
                        <th>Is in WHERE</th>
                        <th>Operation</th>
                        <th>Value</th>
                    </tr>
                </thead>
                <tbody>
                    {{ range .SelectColumns }}
                    <tr>
                        <td><strong>{{ .Name }}</strong> <small>({{
                        ↪  .DataType }})</small></td>
                        <td>
                            <input type="checkbox" name="inc_{{ .Name }}"
                            ↪  value="1" checked>
                        </td>
                        <td>
                            <input type="checkbox" name="where_en_{{ .Name
                            ↪  }}" value="1">
                        </td>
```

Листинг 25 — Интерактивный конструктор выборки в `templates/select.html`

Обработка запроса выполняется в функции `SelectHandler` (листинг 26).

```
101                 whereClause := ""
102                 if len(whereParts) > 0 {
103                         whereClause = " WHERE " + strings.Join(whereParts,
                            ↪  " AND ")
104                 }
105
106                 orderByCol := strings.TrimSpace(r.FormValue("order_by"))
107                 orderDir := strings.TrimSpace(r.FormValue("order_dir"))
108                 if orderDir != "DESC" {
109                         orderDir = "ASC"
110                 }
111                 orderClause := ""
112                 if orderByCol != "" && orderByCol != "none" {
113                         orderClause = fmt.Sprintf(" ORDER BY %s %s",
                            ↪  quoteIdent(orderByCol), orderDir)
114                 }
115
116                 querySQL := fmt.Sprintf("SELECT %s FROM %s%s%s;",
                    ↪  selectList, quoteIdent(table), whereClause,
                    ↪  orderClause)
117
118                 rows, err := pool.Query(querySQL)
119                 if err != nil {
120                         data.Status = "error"
121                         data.Message = "Failed to execute SELECT: " +
                            ↪  err.Error()
122                         data.SQL = querySQL
123                         return views.Render(w, "select", data)
124                 }
125                 defer rows.Close()
```

Листинг 26 — Сборка условий фильтрации, сортировки и запроса в `handlers_query.go`

Результат запроса считывается методом `pool.Query(querySQL)`, оборачивается в структуру `TableData` и отображается пользователю.

3.5.2 Внутреннее соединение таблиц: POST /query/join

Соединение таблиц по маршруту `/join` демонстрирует операцию внутреннего соединения (`INNER JOIN`). В качестве демонстрационной схемы используются таблицы `lab2_employee_ilinov` и `departments`, связанные отно-

шением внешнего ключа: столбец `department_id` таблицы сотрудников ссылается на первичный ключ `id` таблицы отделов.

Обработчик `JoinHandler` выполняет валидацию и собирает итоговый запрос (листинг 27).

```
235                 querySQL := fmt.Sprintf("SELECT %s\nFROM %s AS t1\nINNER
                    ↪  JOIN %s AS t2\n    ON t1.%s = t2.%s;",
236                         strings.Join(selectItems, ", "),
237                         quoteIdent(t1),
238                         quoteIdent(t2),
239                         quoteIdent(col1),
240                         quoteIdent(col2))
241
242                 rows, err := pool.Query(querySQL)
243                 if err != nil {
244                         data.Status = "error"
245                         data.Message = "Failed to execute INNER JOIN: " +
                            ↪  err.Error()
246                         data.SQL = querySQL
247                         return views.Render(w, "join", data)
248                 }
249                 defer rows.Close()
```

Листинг 27 — Сборка и исполнение команды `INNER JOIN` в `handlers_query.go`

3.6 Механизм сброса и начальной инициализации схемы

Поскольку лабораторная работа предполагает активное тестирование деструктивных операций DDL и DML, в приложении предусмотрен механизм изоляции и быстрого восстановления исходного состояния базы данных.

3.6.1 Начальная инициализация схемы базы данных

При старте сервера функция `main` вызывает процедуру `initDatabaseTables` (листинг 28).

```
48 func initDatabaseTables(pool *pgx.ConnPool) error {
49         if _, err := pool.Exec(schemaSQL); err != nil {
50                 return fmt.Errorf("failed to execute schema SQL: %w", err)
51         }
52
53         var deptCount int
54         if err := pool.QueryRow("SELECT COUNT(*) FROM
           ↪  departments").Scan(&deptCount); err != nil {
55                 return err
56         }
57
58         if deptCount == 0 {
59                 if _, err := pool.Exec(seedDepartmentsSQL); err != nil {
60                         return fmt.Errorf("failed to seed departments: %w",
                           ↪  err)
61                 }
62         }
63
64         var empCount int
65         if err := pool.QueryRow("SELECT COUNT(*) FROM
           ↪  lab2_employee_ilinov").Scan(&empCount); err != nil {
66                 return err
67         }
68
69         if empCount == 0 {
70                 if _, err := pool.Exec(seedEmployeesSQL); err != nil {
71                         return fmt.Errorf("failed to seed employees: %w",
                           ↪  err)
72                 }
73         }
74
75         return nil
76 }
```

Листинг 28 — Инициализация таблиц и первичное наполнение данными в `db.go`

Исходные скрипты встроены в бинарный исполняемый файл с помощью директивы `//go:embed`. Функция выполняет следующие шаги:

а) Создает таблицы `departments` и `lab2_employee_ilinov` с помощью скрипта `schema.sql` (если они еще не созданы);

б) Выполняет проверку количества строк (`SELECT COUNT(*)`) в каждой из таблиц;

в) При отсутствии записей наполняет таблицу отделов 5 строками (`seed_departments.sql`), а затем связанную по внешнему ключу `department_id` таблицу сотрудников 20 строками (`seed_employees.sql`).

3.6.2 Сброс POST /reset

Кнопка `Reset` в шапке веб-интерфейса отправляет запрос на эндпоинт `POST /reset`, активируя обработчик `ResetHandler` (листинг 29).

```
42 func ResetHandler(views *ViewRenderer, pool *pgx.ConnPool) AppHandler {
43         return func(w http.ResponseWriter, r *http.Request) error {
44                 if err := resetDatabaseTables(pool); err != nil {
45                         base, bErr := newBasePage(pool)
46                         if bErr == nil {
47                                 base.Status = "error"
48                                 base.Message = "Database reset failed: " +
                                    ↪ err.Error()
49                                 return views.RenderStatus(w, "tables",
                                    ↪ http.StatusInternalServerError,
                                    ↪ TablesPageData{BasePage: base,
                                    ↪ ShowViewAll: true})
50                         }
51                         return &AppError{Err: err, Message: "Database reset
                           ↪ failed: " + err.Error(), Code:
                           ↪ http.StatusInternalServerError}
52                 }
53                 redirectSuccess(w, r, "/tables")
54                 return nil
55         }
56 }
```

Листинг 29 — Обработчик сброса базы данных в `handlers_tables.go`

Обработчик вызывает функцию `resetDatabaseTables` (листинг 30).

```
78 func resetDatabaseTables(pool *pgx.ConnPool) error {
79         tables, err := getTables(pool)
80         if err != nil {
81                 return err
82         }
83         for _, t := range tables {
84                 _, _ = pool.Exec(fmt.Sprintf("DROP TABLE IF EXISTS %s
                    ↪  CASCADE", quoteIdent(t)))
85         }
86         return initDatabaseTables(pool)
87 }
```

Листинг 30 — Каскадное удаление всех таблиц и повторная инициализация

3.7 Результаты работы и снимки экрана приложения

Для подтверждения корректности работы разработанного веб-сервера и взаимодействия с СУБД PostgreSQL было проведено комплексное тестирование всех реализованных сценариев.

Снимок экрана страницы просмотра таблиц схемы представлен на рисунке 3.1.

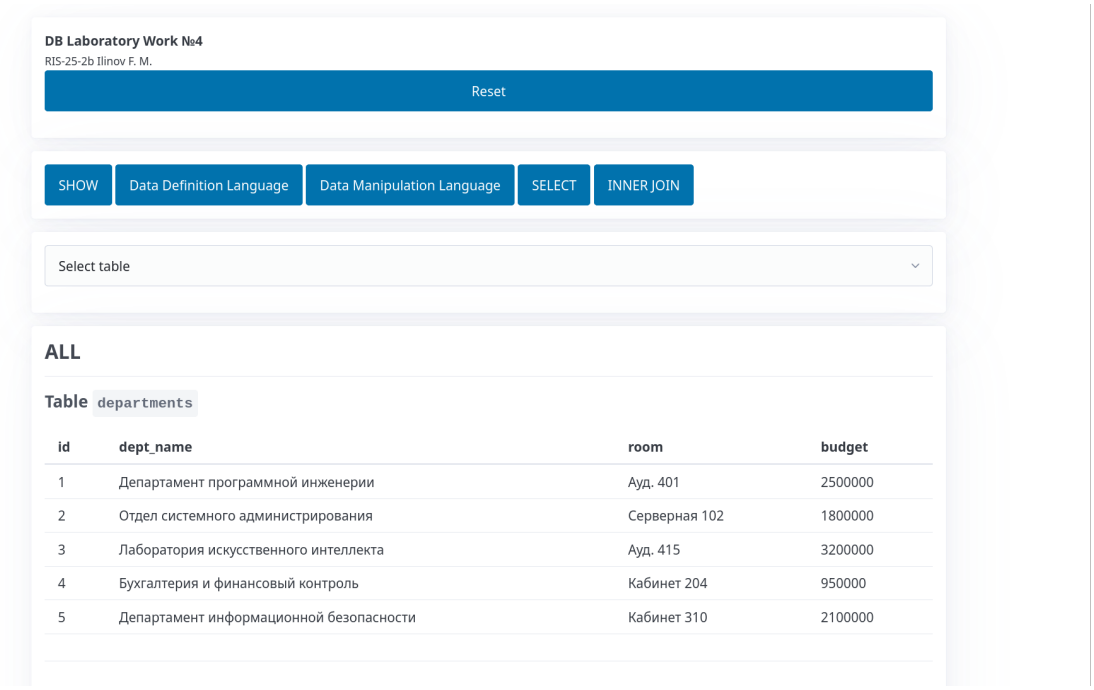


Рисунок 3.1 — Отображение списка таблиц и их кортежей в браузере

Снимок экрана создания новой таблицы с первичным и внешним ключами приведен на рисунке 3.2.

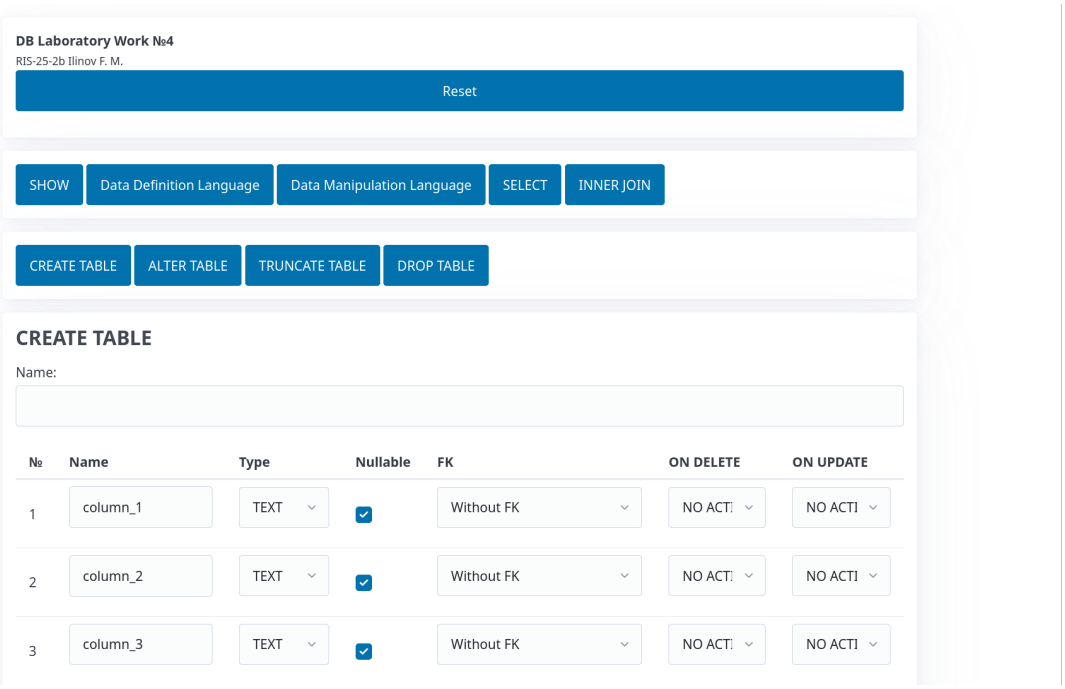


Рисунок 3.2 — Создание таблицы через двухэтапную форму DDL

Снимок экрана модификации колонок таблицы с помощью команды ALTER TABLE приведен на рисунке 3.3.

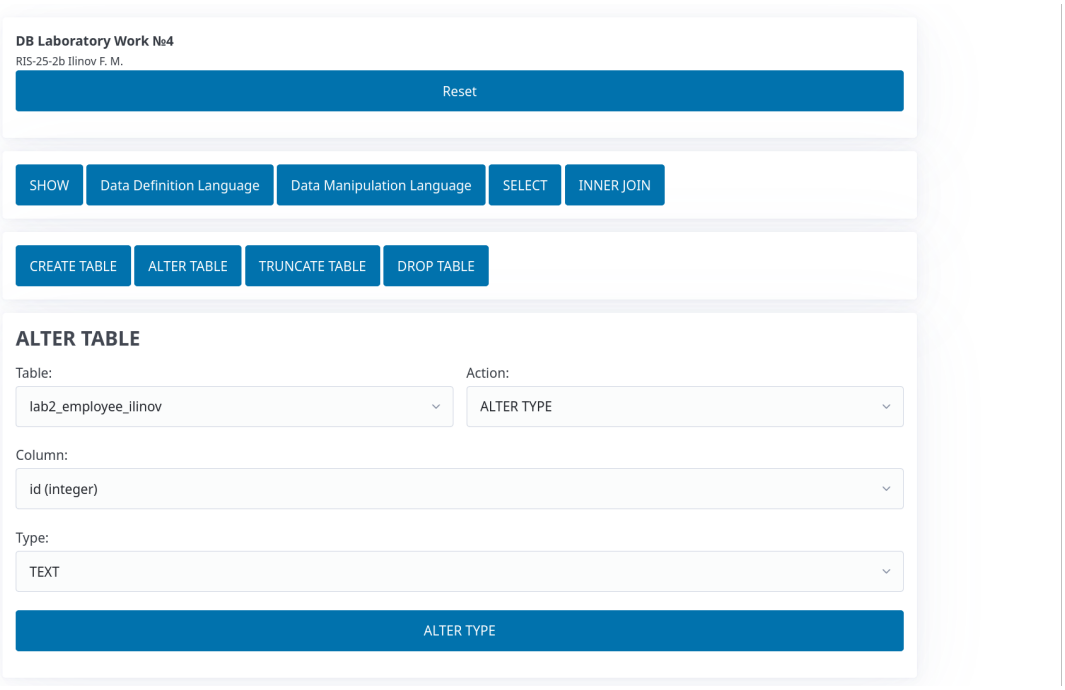


Рисунок 3.3 — Модификация структуры таблицы с помощью команды ALTER TABLE

Снимок экрана очистки данных таблицы оператором TRUNCATE представлен на рисунке 3.4.

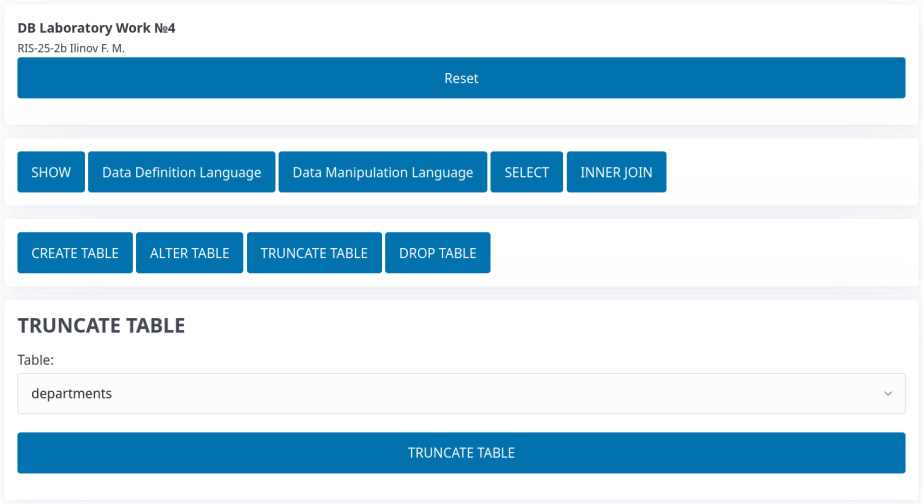


Рисунок 3.4 — Очистка таблицы оператором TRUNCATE TABLE CASCADE

Снимок экрана удаления таблицы с каскадной очисткой связей приведен на рисунке 3.5.

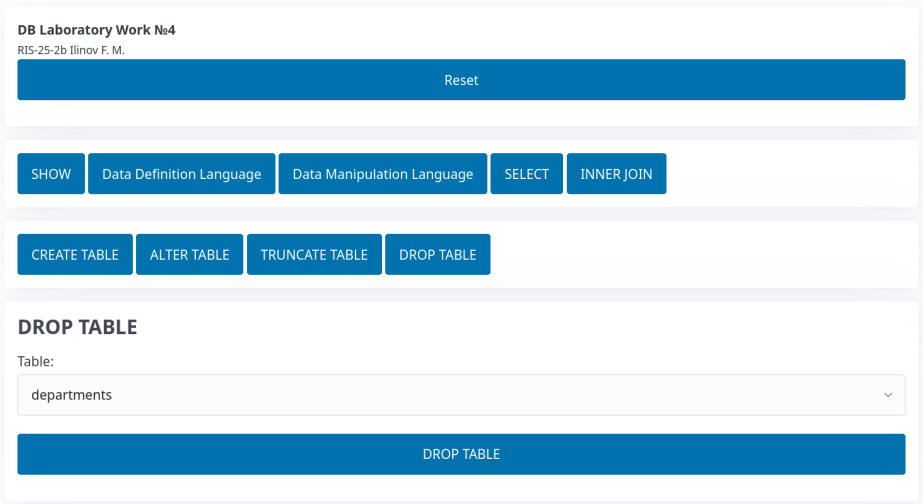


Рисунок 3.5 — Удаление сущности оператором DROP TABLE CASCADE

Снимок экрана пакетного добавления нескольких кортежей оператором INSERT INTO представлен на рисунке 3.6.

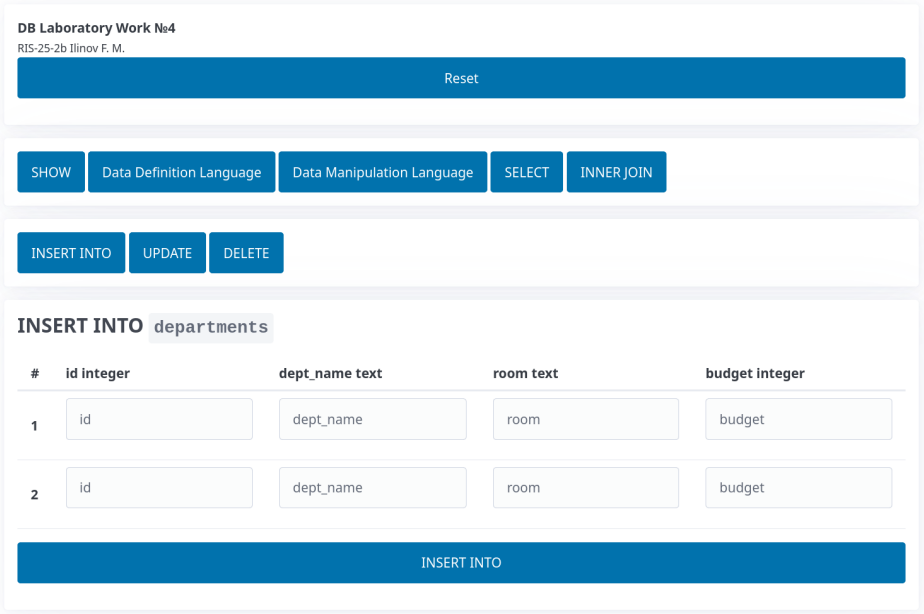


Рисунок 3.6 — Пакетная вставка записей с помощью команды INSERT INTO

Снимок экрана изменения данных оператором UPDATE с предикатом WHERE приведен на рисунке 3.7.

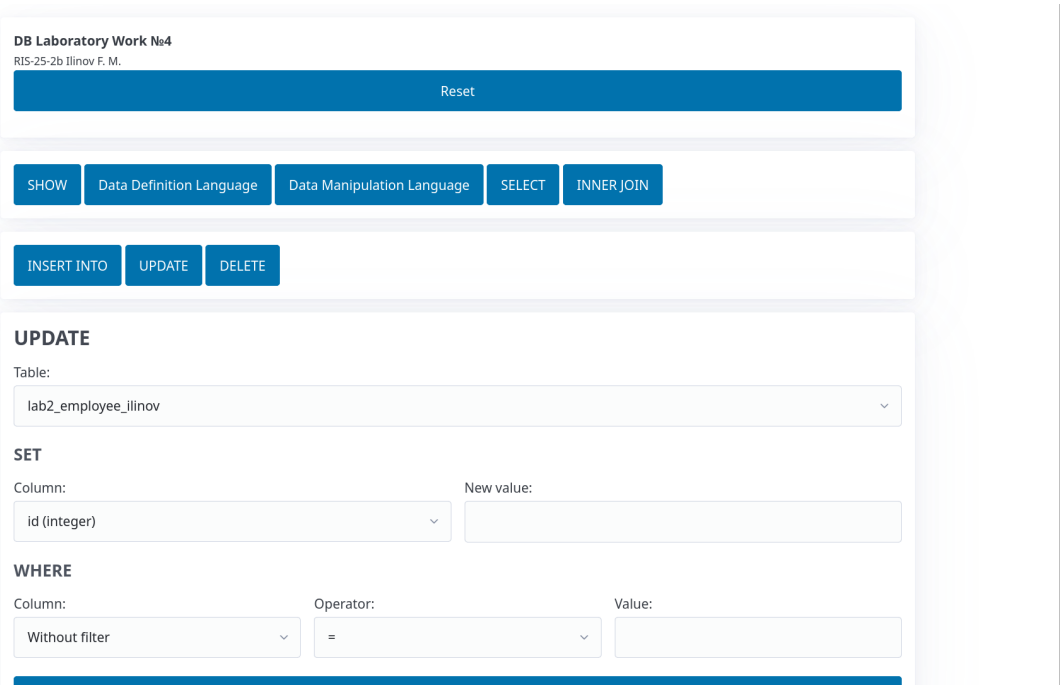


Рисунок 3.7 — Обновление данных с условием фильтрации

Снимок экрана удаления записей оператором DELETE приведен на рисунке 3.8.

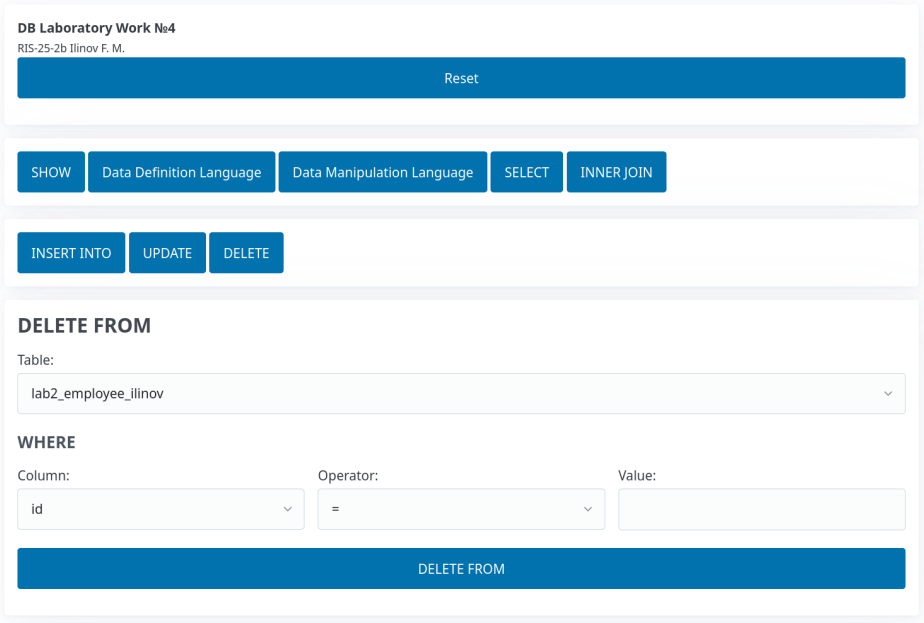


Рисунок 3.8 — Удаление записей оператором DELETE FROM по условию

Снимок экрана параметризованной выборки SELECT с фильтрацией и сортировкой приведен на рисунке 3.9.

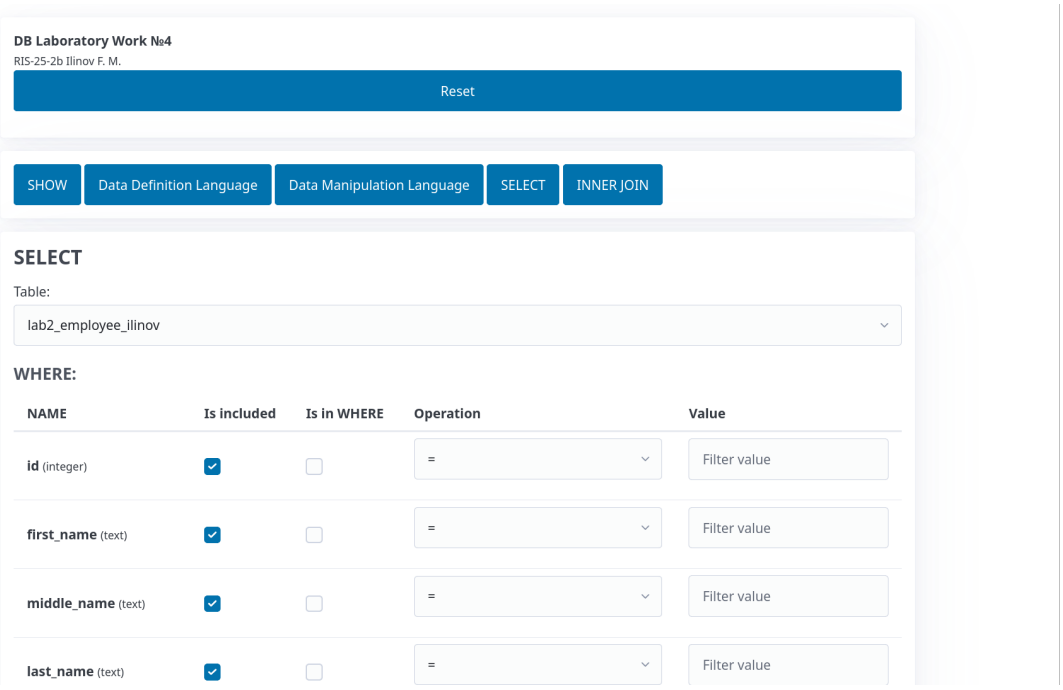


Рисунок 3.9 — Результаты параметризованной выборки SELECT ... WHERE

Снимок экрана внутреннего соединения двух таблиц оператором INNER JOIN представлен на рисунке 3.10.

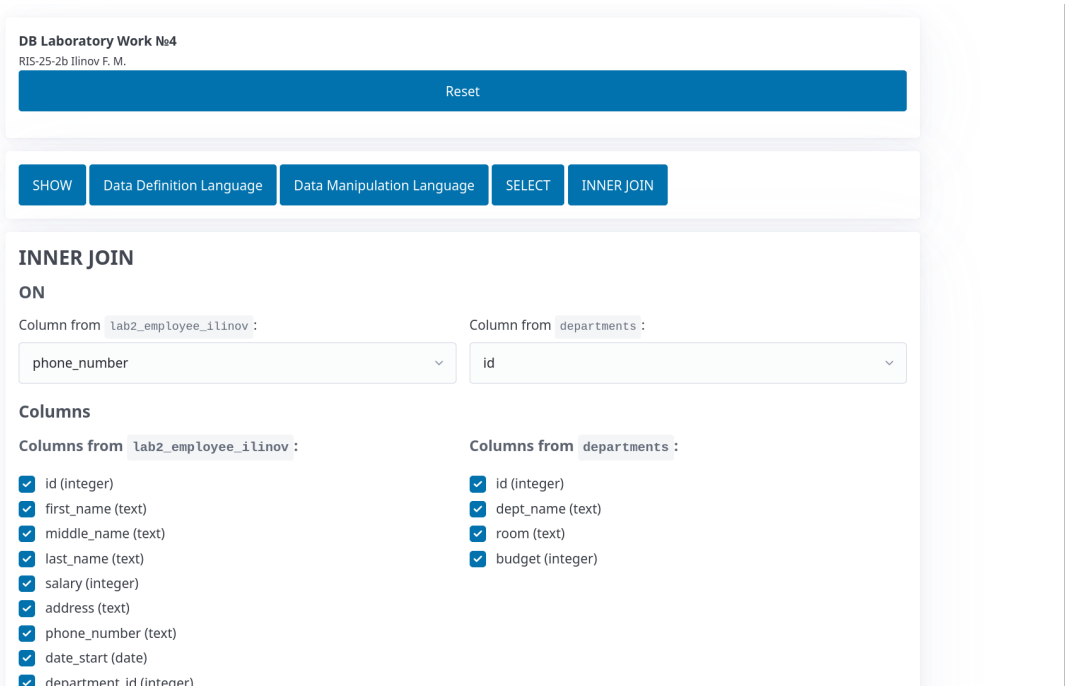


Рисунок 3.10 — Результаты реляционного соединения INNER JOIN ... ON

На всех этапах тестирования сформированные SQL-запросы корректно валидировались и исполнялись СУБД PostgreSQL, а результирующие наборы данных и системные уведомления в полном объеме выводились в веб-интерфейсе.

ЗАКЛЮЧЕНИЕ

В ходе работы были решены следующие задачи:

а) Разработан веб-сервер с модульной архитектурой и сквозным стеком вызовов от пользовательских HTML-форм до ядра СУБД PostgreSQL;

б) Реализовано безопасное конструирование SQL-запросов с экранированием идентификаторов (`quoteIdent`) и литералов (`quoteLiteral`);

в) Реализована поддержка операций языка определения данных DDL: двухэтапное создание таблиц с поддержкой первичных и внешних ключей (включая самоссылающиеся ограничения), изменение структуры через ALTER TABLE, очистка через TRUNCATE и каскадное удаление через DROP TABLE CASCADE;

г) Реализована поддержка операций языка манипулирования данными DML: пакетная вставка записей с автоматической обработкой NULL-значений, изменение данных по условию UPDATE и удаление данных DELETE;

д) Реализован интерактивный конструктор запросов выборки SELECT с динамической проекцией, фильтрацией и сортировкой, а также оператор реляционного соединения INNER JOIN двух таблиц по общему ключу;

е) Реализован механизм изоляции и быстрого восстановления исходного состояния базы данных через эндпоинт `POST /reset`.

Корректность функционирования подтверждена серией тестов в браузере с прозрачным аудитом текста сформированных SQL-команд.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. PostgreSQL 18.6 Documentation. The PostgreSQL Global Development Group. — 2026. — Режим доступа: https://www.postgresql.org/files/documentation/pdf/18/postgresql-18-A4.pdf (дата обращения: 01.09.2026).

2. ISO/IEC. ISO/IEC 9075:2023 Information technology — Database languages — SQL. — 2023. — Режим доступа: https://www.iso.org/standard/76583.html (дата обращения: 01.09.2026).

3. Mozilla. HTTP: Hypertext Transfer Protocol. — 2026. — Режим доступа: https://developer.mozilla.org/en-US/docs/Web/HTTP (дата обращения: 01.09.2026).

4. Mozilla. HTML: HyperText Markup Language. — 2026. — Режим доступа: https://developer.mozilla.org/en-US/docs/Web/HTML (дата обращения: 01.09.2026).

5. Larroche Lucas. Pico CSS • Minimal CSS Framework for semantic HTML. — 2026. — Режим доступа: https://picocss.com/ (дата обращения: 01.09.2026).

6. Google. The Go Programming Language (version 1.27). — 2026. — Режим доступа: https://go.dev/ (дата обращения: 01.09.2026).

7. jackc. pgx - PostgreSQL Driver and Toolkit. — 2026. — Режим доступа: https://github.com/jackc/pgx (дата обращения: 01.09.2026).